

ABC 282 补题报告

D题 ps: 附上 知乎题解

这题有点超出能力范围，几个概念：二分图，连通分量，染色法。

对于有多个连通分量的图来说：两个不同连通分量上的任意点相连还是二分图。

思路：二分图等效于没有奇数环(画图就能证明)，经过手算样例，可知此题为求最多再连接多少条边，这个图还是一个二分图。那么我们可以使用染色法，如果通过染色法,图不是二分图,答案一定为0；还有一点很重要图不一定连通。

假设我们此时遍历的 cnt 个点是一个二分图，其中白色点数目为 w ，黑色点数目为 b ，有 cntt 条边，那么还可以连接 $w*b-\text{cntt}$ 条边。

此外,对于剩下不与这个二分图相连的点,二分图的点,都可以与其连接， $\text{cnt}*(w-\text{cnt})$ ， w 初值为 n 且 w 要变化,因为这 cnt 个点已经不用考虑了,所以 $w-=\text{cnt}$,避免遍历剩下的点时对之前的点造成影响。

还可以在用dfs跑二分图染色时返回连通分量上黑点与白点的个数

[另外一个blog](#)

```
#include<bits/stdc++.h>
#define int long long
using namespace std;
typedef pair<int,int> PLL;
set<PLL> st;
constexpr int N=2e5+30;
vector<int> g[N];
int n,m;
bool flag=true;
int cnt;
int color[3];
int a[N];
void dfs(int cur,int fa,int x)
{
    a[cur]=x;
    color[x]++;
    cnt++;
    for(auto k:g[cur])
    {
        if(k==fa) continue;
        st.insert({k,cur});
        st.insert({cur,k}); //双向存
        if(a[k])
        {
            if(a[cur]==a[k]) flag=false;
        }
        else
        {
            dfs(k,cur,3-x);
        }
    }
}
```

```

}
signed main()
{
    scanf("%lld%lld",&n,&m);
    for(int i=1;i<=m;++i)
    {
        int u,v;
        cin>>u>>v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    int ans=0;
    flag=true;
    int val=n;
    for(int i=1;i<=n;++i)
    {
        if(!a[i])
        {
            st.clear();
            cnt=0;
            color[1]=0;
            color[2]=0;
            dfs(i,i,1);
            int cntt=st.size()/2;
            ans+=color[1]*color[2]-cntt;
            ans+=cnt*(val-cnt);
            val-=cnt;
        }
    }
    if(!flag) ans=0;
    printf("%lld\n",ans);
    return 0;
}

```

E题

因为每次气球实际上是消耗了一个，另一个会被保留下来，所以取值可以看作是两点之间边的权重，从n个顶点中选出n-1条边使得权重最大，相当于求最大生成树，思路及代码参考[blog](#)

```

#include <iostream>
#include <algorithm>
#include <cstring>
using namespace std;
typedef long long ll;
const int N = 502;
struct node
{
    int len;
    int from, to;
    bool operator<(const node &t) const
    {
        return len > t.len;
    }
} sides[N * N];
int arr[N], n, m, pos = 0;

```

```

int quick(int a, int b, int m)
{
    int ans = 1;
    while (b)
    {
        if (b & 1)
            ans = ((ll)ans * a) % m;
        b >>= 1;
        a = ((ll)a * a) % m;
    }
    return ans;
}

int h[N];
int find(int x)
{
    if (h[x] != x)
        h[x] = find(h[x]);
    return h[x];
}

ll kruskal()
{
    int count = 1;
    ll ans = 0;
    for (int i = 1; i <= n; i++)
        h[i] = i;
    for (int i = 1; i <= pos; i++)
    {
        int len = sides[i].len;
        int from = sides[i].from, to = sides[i].to;
        int ff = find(from), ft = find(to);
        if (ff == ft)
            continue;
        else
        {
            count++;
            h[ft] = ff;
            ans += len;
            if (count == n)
                break;
        }
    }
    return ans;
}

int main()
{
    cin >> n >> m;
    for (int i = 1; i <= n; i++)
        scanf("%d", &arr[i]);
    // 无向图的最大生成树
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
        {
            if (i != j)
            {
                int x = arr[i], y = arr[j];

```

```
        int len = ((ll)quick(x, y, m) + quick(y, x, m)) % m;
        sides[++pos] = {len, i, j};
    }
}
sort(sides + 1, sides + pos + 1);
cout << kruskal();
return 0;
}
```